Baja SAE IMU Data Logger — Full Build & Replication Guide

Project: Raspberry Pi 4B + WitMotion WT901 IMU logging system
Purpose: In-vehicle accelerometer / gyroscope / angle / magnetometer data capture with GPIO-controlled start/stop buttons and LED indicators, auto-starting on boot via systemd.
Status as documented: Fully operational. First deployed March 2026.
Target audience: Baja team members replicating or extending this system from scratch.

> **No internet required. This system operates completely offline once built. There is no cloud dependency, no API key, no WiFi needed in the field. The Pi logs to its SD card. To retrieve data after a run, either SSH in over a local network and `scp` the files off, or pull the SD card and mount it directly on any computer using AnyLinuxFS (macOS/Windows) or native Linux filesystem tools. Log files live at `/home/m1000/wt901/witmotion_raw.log` and parsed CSVs in the same directory. If you parse a raw log, it will return the parsed csv with timestamps.**

Table of Contents

1. Hardware List & Wiring
2. Raspberry Pi OS Setup
3. Serial Port Configuration
4. Python Virtual Environment & Dependencies
5. Project File Structure
6. Software Architecture Overview
7. The Bootloader (`bootloadervw.py`)
8. The IMU Logger (`imu_logger2.1.py`)
9. The Shell Wrapper (`run_bootloader.sh`)
10. systemd Services
11. Data Parsing (`witmotion_log_parser-2.py`)
12. Sensor Mount & Axis Correction
13. Troubleshooting Log — Problems Solved
14. Calibration Procedure
15. Testing the System
16. Future Improvements Roadmap
17. Quick Reference Cheat Sheet

---

1. Hardware List & Wiring

Components

Item	Notes
Raspberry Pi 4B (8 GB RAM)	Any 4B variant works; 8 GB is comfortable headroom
WitMotion WT901 IMU	9-axis: accelerometer, gyroscope, magnetometer + onboard angle fusion
GPIO terminal/header breakout board	Simplifies reliable wiring vs. bare header pins
	Boot indicator and armed/logging indicator
Momentary push buttons (2–3x)	START, STOP, and optionally a dedicated safe-shutdown button
Serial wiring	3-wire: TX, RX, GND between Pi and WT901
Micro-SD card (16 GB+)	Class 10 or better
5V power supply (3A minimum)	Underpowering causes random crashes — do not cheap out here

GPIO Pin Assignments (BCM numbering)

Signal	BCM Pin	Notes
BOOT_LED	GPIO 26	Solid on = bootloader is alive and running
ARMED_LED	GPIO 19	Blinks at 1 Hz = ready and waiting for START button
LOG_LED	GPIO 21	Blinks at 4 Hz = actively logging IMU data
SHUTDOWN_LED	GPIO 18	Blinks at 10 Hz during graceful shutdown sequence
START button	GPIO 13	Active LOW (internal pull-up enabled in
STOP button	GPIO 17	Active LOW (internal pull-up enabled in
SHUTDOWN	GPIO 16	Active LOW — triggers graceful OS shutdown
WT901 TX → Pi RX	GPIO 15 (RXD)	**Yellow wire** from WT901 TX
WT901 RX → Pi TX	GPIO 14 (TXD)	**Green wire** to WT901 RX
GND	Any GND	**Purple wire** — shared ground to WT901
5V power	5V pin	**Red wire** — VCC to WT901

LED Behavior Summary

Understanding the LED states at a glance is critical in the field:

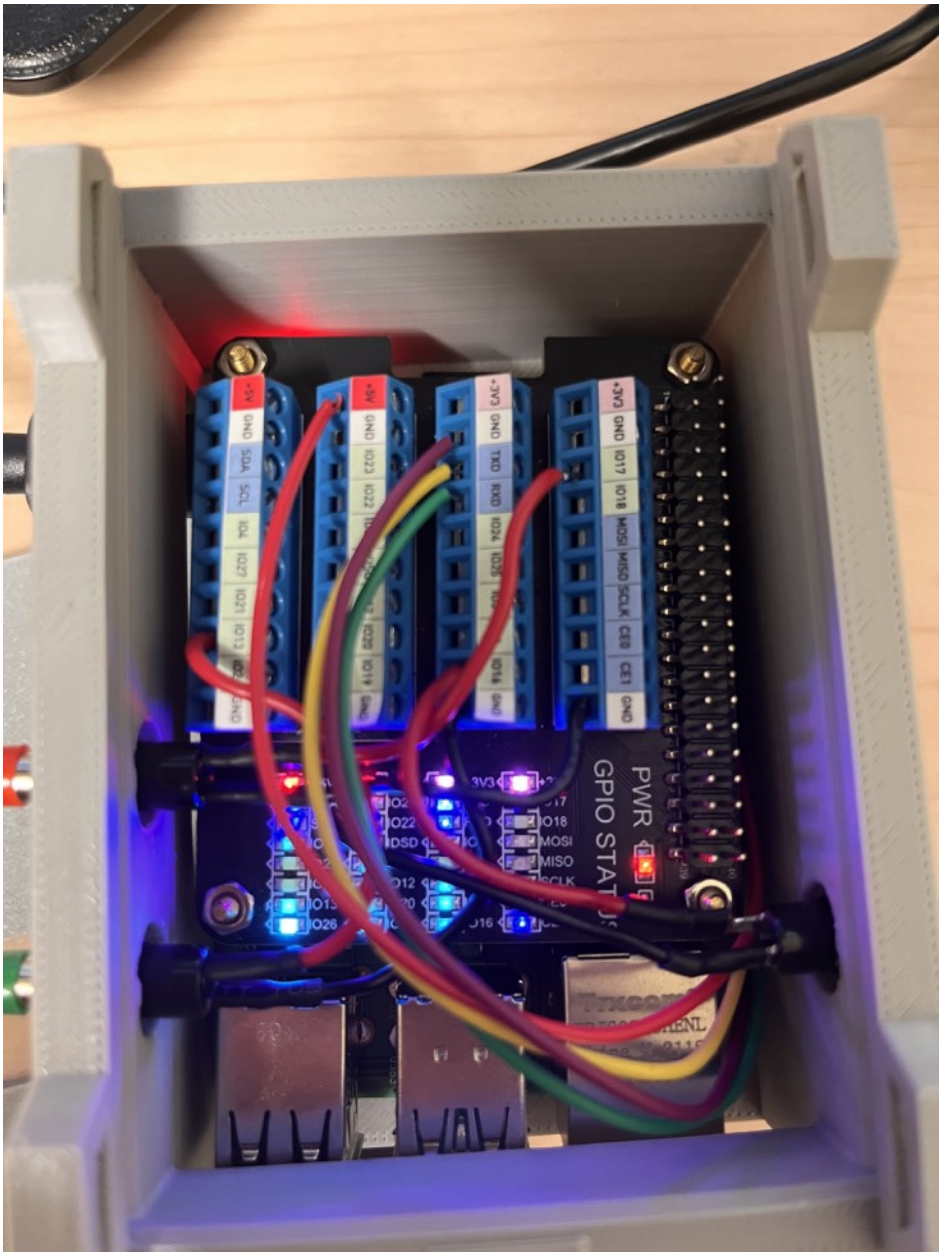
LED	GPIO	Pattern	Meaning
BOOT	26	Solid ON	System is booted and bootloader is running
ARMED	19	1 Hz blink	Waiting for you to press START
LOG	21	4 Hz blink	Actively recording IMU data
SHUTDOWN	18	10 Hz blink	Shutdown sequence in progress — wait for it to finish before cutting power

Always wait for all LEDs to go dark before removing power. The 10 Hz shutdown blink on GPIO 18 is your signal that the OS is in the process of halting. Cutting power during this window risks SD card corruption.

Wiring Notes

The enclosure uses a 3D-printed case with a GPIO terminal/screw-block breakout board mounted on top of the Pi. This board exposes every GPIO as a labeled screw terminal, which makes field wiring and re-wiring far more reliable than bare header pins. Strongly recommended for any vehicle-mounted build.

- **WT901 power:** The WT901 is powered from the Pi's 5V pin via the red wire. This is fine for bench use. In the car, consider whether the Pi's 5V rail is stable enough under vibration and load — if you see IMU dropouts, power the WT901 from a separate regulated supply.
- **WT901 logic levels:** The WT901 communicates at 3.3V logic over serial. The Pi's UART TX/RX are also 3.3V. No level shifter is needed. Do **not** connect the WT901 RX pin to a 5V signal — it will damage the sensor.
- Buttons: Wire each button between its GPIO pin and any GND terminal. The Pi's internal pull-up resistors are enabled in software — no external resistors are needed for the buttons.



---

2. Raspberry Pi OS Setup

Flash the OS

Use the Raspberry Pi Imager tool (https://rpi.imager.org) to flash Raspberry Pi OS FULL (64-bit) to your SD card.

In the Imager's advanced settings (gear icon) before flashing:

- Set hostname (e.g., `baja-logger`) (M1000 PREFERRED FOR PATHS)
- Enable SSH

- Set your username (this guide uses `m1000` — adjust all paths if you use a different username)
- Set WiFi credentials (useful for first-time setup; can be removed later)
- Set locale/timezone

First Boot & Update

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv git screen
```

Create the project user (if not already `m1000`)

All paths in the scripts are hardcoded to `/home/m1000/wt901/`. If you use a different username, you **must** update the paths in every script and service file. It is easiest to just create this user or adjust everything up front.

```
# If your Pi user is not m1000, create a symlink or find-replace all
paths:
sudo useradd -m -s /bin/bash m1000
sudo usermod -aG gpio,dialout m1000
```

If you keep your own username, add yourself to the required groups:

```
sudo usermod -aG gpio,dialout $USER
# Log out and back in for group changes to take effect
```

---

3. Serial Port Configuration

This is one of the most critical and error-prone steps. The Pi has two UARTs: a full hardware UART (PL011) and a mini-UART. By default, the mini-UART is assigned to `ttyS0` and used for the serial console. The WT901 **requires** the full hardware UART for reliable communication.

Step 1 — Disable the serial console

```
sudo raspi-config
```

Navigate to: **Interface Options → Serial Port**

- "Would you like a login shell accessible over serial?" → **No**
- "Would you like the serial port hardware to be enabled?" → **Yes**

This disables the console on the serial pins but keeps the hardware port active.

Step 2 — Disable Bluetooth to free the full UART

The Pi 4's PL011 (full hardware UART) is assigned to Bluetooth by default. To reclaim it for serial use, add the following to `/boot/config.txt` (or `/boot/firmware/config.txt` on newer OS versions):

```
dtoverlay=disable-bt
```

Then disable the Bluetooth service:

```
sudo systemctl disable hciuart
sudo systemctl disable bluetooth
```

Reboot: `sudo reboot`

Step 3 — Identify your serial port

After reboot:

```
ls -la /dev/serial* /dev/ttyAMA* /dev/ttyS*
```

You should see `/dev/ttyAMA0` pointing to the hardware UART. The working port in this build is:

- **Primary:** `/dev/ttyAMA0` (used in `imu_logger2.1.py`)
- **Alternate:** `/dev/ttyS0` (used in `imu_logger4_API.py` — verify which port your system assigns)

Test that the port exists and you have access:

```
ls -la /dev/ttyAMA0
# Should show: crw-rw---- 1 root dialout ...
# Your user must be in the 'dialout' group
```

Step 4 — Verify baud rate

The WT901's default baud rate from the factory is **9600 baud**. This is what the system uses. Do not change it in the WT901 firmware unless you explicitly need higher throughput, because changing it incorrectly will make the device appear dead (see "Bricking the Device" in the Troubleshooting section).

---

4. Python Virtual Environment & Dependencies

A virtual environment (`venv`) is essential here. System Python on Raspberry Pi OS is managed by the OS package manager, and installing packages globally with pip will fail or conflict on modern Debian-based systems.

Create the venv

```
mkdir -p /home/m1000/wt901
cd /home/m1000/wt901
python3 -m venv .
# This creates bin/, lib/, include/ inside /home/m1000/wt901/
```

The venv Python binary is now at `/home/m1000/wt901/bin/python`. This exact path is hardcoded in the bootloader and shell scripts — do not change the venv location.

Install dependencies

```
/home/m1000/wt901/bin/pip install witmotion RPi.GPIO
```

The `witmotion` package is the community Python library for WitMotion sensors. It wraps the serial protocol and provides message callbacks. The `RPi.GPIO` package controls GPIO pins.

Verify the install:

```
/home/m1000/wt901/bin/python -c "import witmotion; import RPi.GPIO;
print('OK')"
```

Install the witmotion-debug CLI tool

The `imu_logger2.1.py` script uses a command-line binary called `witmotion-debug`. This is installed as part of the witmotion Python package and should appear at:

```
/home/m1000/wt901/bin/witmotion-debug
```

Verify it exists:

```
ls -la /home/m1000/wt901/bin/witmotion-debug
```

Test it manually (Ctrl+C to stop):

```
/home/m1000/wt901/bin/witmotion-debug --path /dev/ttyAMA0 --verbose
```

You should see a stream of lines like:

```
DEBUG:witmotion:state: idle -> header, got 0x55
DEBUG:witmotion:state: header -> payload, got code 0x51
INFO:witmotion.cmd.debug:acceleration message - vec:(0.129, -7.675,
-8.053) temp_celsius:35.28
```

If you see nothing or garbage characters, your serial port or baud rate is wrong (see Troubleshooting).

---

5. Project File Structure

```
/home/m1000/wt901/
├── bin/                          # venv binaries (python, pip,
witmotion-debug, etc.)
├── lib/                          # venv libraries
├── include/                      # venv headers
├── bootloadervw.py               # GPIO supervisor — waits for
button, launches logger
├── imu_logger2.1.py              # Core logger — reads IMU, writes
CSV and raw log
├── imu_logger4_API.py            # Alternative logger using witmotion
Python API directly
```

```
├── run_bootloader.sh              # systemd wrapper script — sets
PATH, launches bootloader
├── witmotion_log_parser-2.py      # Post-run parser — converts raw log
to clean CSV
├── witmotion_raw.log              # Raw log output (appended each
session)
└── witmotion_parsed.csv           # Parsed CSV output (appended each
session)

/etc/systemd/system/
├── wt901_bootloader.service       # Auto-starts the bootloader on boot
└── shutdown-monitor.service       # Monitors a GPIO button for safe
shutdown
```

---

6. Software Architecture Overview

The system has a layered design with clear separation of concerns:

```
systemd
  └── wt901_bootloader.service
        └── run_bootloader.sh          (sets environment, launches
with venv Python)
              └── bootloadervw.py      (GPIO supervisor)
                    └── imu_logger2.1.py  (spawned as subprocess when
START is pressed)
                          └── witmotion-debug CLI  (spawned as
subprocess, streams IMU data)
```

Why this layered approach?

- `systemd` manages auto-start, restart on failure, and logging to journald.
- `run_bootloader.sh` solves the PATH problem: systemd runs in a minimal environment and cannot find the venv Python without explicit path setting.
- `bootloadervw.py` runs continuously, managing the LED states and waiting for user input. It does GPIO cleanup and re-init between sessions so the logger starts fresh.
- `imu_logger2.1.py` is a child process. When the STOP button is pressed, the logger exits cleanly, and the bootloader returns to its waiting state — ready for the next run without requiring a reboot.
- The logger itself spawns `witmotion-debug` as a subprocess and reads its stdout line-by-line, parsing and writing to CSV.

Data flow:

```
WT901 IMU (serial) → witmotion-debug CLI → stdout pipe
                                                ↓
                                        imu_logger2.1.py
```

```
                                        ├── witmotion_raw.log    (every
line, raw)
                                        └── witmotion_parsed.csv
(parsed frames)
                                                  ↓ (post-run)
                                    witmotion_log_parser-2.py
                                        └──
<name>_parsed_<timestamp>.csv (clean, corrected)
```

---

7. The Bootloader (`bootloadervw.py`)

The bootloader is the persistent supervisor process. It starts on boot and never exits unless the Pi is shut down.

Behavior:

1. Turns on BOOT_LED (solid) to indicate the system is alive.
2. Blinks ARMED_LED at ~1 Hz, waiting for the START button.
3. On START press: turns off ARMED_LED, calls `GPIO.cleanup()` to release all pins, then spawns `imu_logger2.1.py` as a subprocess and **blocks** until the logger exits.
4. After the logger exits: re-initializes GPIO and returns to step 2.

Why `GPIO.cleanup()` before spawning the logger?

The logger also uses GPIO. If the bootloader holds the GPIO resources when the logger starts, you get permission errors or undefined pin states. Cleanup releases the pins so the logger can claim them cleanly. After the logger exits, the bootloader re-initializes its own pins.

Key code excerpt:

```python
PYTHON_BIN = "/home/m1000/wt901/bin/python"
LOGGER_PATH = "/home/m1000/wt901/imu_logger2.1.py"

GPIO.cleanup()  # Release pins before spawning child
logger_process = subprocess.Popen([PYTHON_BIN, LOGGER_PATH])
logger_process.wait()  # BLOCK until logger exits
```

Why hardcode the venv Python path?

When spawned by systemd, the `PATH` environment variable is minimal. Using `/usr/bin/python3` would use the system Python, which doesn't have `witmotion` or `RPi.GPIO` installed in the venv. Hardcoding the venv binary ensures the correct interpreter and packages are always used.

---

8. The IMU Logger (`imu_logger2.1.py`)

The logger is the core data capture process. It is designed to be short-lived — one run per button press/release cycle.

Startup sequence:

1. Sets up GPIO pins (START, STOP, LOG_LED).
2. Opens raw log and CSV files in append mode.
3. Waits for the START button press.
4. Spawns `witmotion-debug` as a subprocess, capturing stdout.
5. Enters the main loop.

Main loop:

- Toggles LOG_LED at ~4 Hz (fast blink = actively logging).
- Checks STOP button — exits cleanly if pressed.
- Uses `select.select()` for non-blocking reads from the subprocess stdout. This avoids blocking the button check and LED blink.
- Writes every raw line to `witmotion_raw.log`.
- Parses lines containing `"payload"` for packet type and data, writes to CSV.

Log file behavior — append mode:

Both files are opened with `"a"` (append). This means each session adds to the existing file. The raw log grows indefinitely; the parser can handle multi-session logs. If you want per-session files, change the open mode to `"w"` or add a timestamp to the filename.

CSV columns written in real-time:

```
timestamp, type, data, checksum_status
```

Note: this is a simplified on-the-fly CSV. The full clean CSV with axis-corrected data, proper column names, and merged frames is produced by the post-run parser.

Serial port used: `/dev/ttyAMA0` at 9600 baud (the default from `witmotion-debug`).

---

9. The Shell Wrapper (`run_bootloader.sh`)

This script exists purely to solve the systemd PATH problem.

```
VENV_PYTHON="/home/m1000/wt901/bin/python"
BOOTLOADER_PY="/home/m1000/wt901/bootloadervw.py"

export PATH="/home/m1000/wt901/bin:$PATH"
echo "$(date) - Starting bootloader via run_bootloader.sh" >>
"$RAW_LOG"
```

```
exec "$VENV_PYTHON" "$BOOTLOADER_PY" >> "$RAW_LOG" 2>&1
```

Key points:

- `export PATH=...` prepends the venv bin directory so any subprocess that looks up executables by name (like `witmotion-debug`) finds the venv version first.
- `exec` replaces the shell process with Python, so systemd tracks the Python PID directly.
- Stdout and stderr are redirected to the raw log, so bootloader startup messages are recorded.
- The `echo "$(date) - Starting..."` line creates visible session boundaries in the log file, making it easy to find where each boot begins.

---

10. systemd Services

`wt901_bootloader.service`

Place this file at `/etc/systemd/system/wt901_bootloader.service`:

```
[Unit]
Description=WT901 IMU Bootloader Supervisor
After=network.target

[Service]
Type=simple
User=m1000
Group=m1000
WorkingDirectory=/home/m1000/wt901
ExecStart=/home/m1000/wt901/run_bootloader.sh
Restart=on-failure
Environment="PATH=/home/m1000/wt901/bin:/usr/bin:/bin"
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Important notes:

- `After=network.target` is listed but not strictly required for IMU logging. It ensures any network-dependent services are up first — useful if you later add telemetry.
- `Restart=on-failure` restarts the bootloader only if it crashes. It does **not** restart it on a clean exit (which is correct — you don't want it restarting during a clean shutdown).

- The duplicate `Restart=` line in the original file (`Restart=always` followed by `Restart=on-failure`) — only the **last** one takes effect. In the current file, that is `on-failure`. Clean this up to avoid confusion.
- `Environment="PATH=..."` is a belt-and-suspenders measure alongside what `run_bootloader.sh` does. Both are needed.

shutdown-monitor.service

Place at `/etc/systemd/system/shutdown-monitor.service`:

```
[Unit]
Description=Shutdown Button Monitor
After=multi-user.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /home/pi/imu-datalogger/shutdown_monitor.py
WorkingDirectory=/home/pi/imu-datalogger
Restart=on-failure
RestartSec=5s
User=pi
Group=pi
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Note: This service still references `/home/pi/imu-datalogger/` and `User=pi`. This is from an earlier version of the project before the username was changed to `m1000`. Before using this service, update the paths and username to match your system.

The `shutdown_monitor.py` script monitors **GPIO 16** for a button press (active LOW). When triggered, it flashes the **GPIO 18 LED at 10 Hz** as a visual warning that shutdown is in progress, then calls `sudo shutdown -h now`. **Always wait for all LEDs to go dark before cutting vehicle power** — removing power during the shutdown flash risks SD card corruption and a broken filesystem.

Enabling and managing the services

```
# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable wt901_bootloader.service
sudo systemctl start wt901_bootloader.service
sudo systemctl enable shutdown-monitor.service
sudo systemctl start shutdown-monitor.service

# Check status
sudo systemctl status wt901_bootloader.service
```

```
# View live logs
sudo journalctl -u wt901_bootloader.service -f

# Stop for debugging
sudo systemctl stop wt901_bootloader.service

# Restart after changes
sudo systemctl restart wt901_bootloader.service
```

---

11. Data Parsing (`witmotion_log_parser-2.py`)

After a logging session, the raw log contains a stream of DEBUG and INFO lines from the `witmotion-debug` tool. The parser converts this into a clean, analysis-ready CSV.

Running the parser

```
# Parse the default raw log
/home/m1000/wt901/bin/python witmotion_log_parser-2.py

# Specify input and output files
/home/m1000/wt901/bin/python witmotion_log_parser-2.py
witmotion_raw.log my_run_parsed.csv
```

What the parser does

- Reads both new-format lines (with `[HH:MM:SS.mmm]` timestamp prefix) and legacy lines (no timestamp).
- Groups acceleration, gyro, angle, and magnetometer messages into complete frames anchored on each acceleration message.
- Applies mount-orientation corrections (see Section 12).
- Writes a CSV with both raw and corrected columns.
- Prints a summary to the terminal: frame count, duration, Hz rate, min/max/avg for all axes.

Output CSV columns

Column	Description
frame	Frame index
timestamp_iso	Full ISO timestamp (if available)
time_hms	HH:MM:SS.mmm
raw_ax_g .. raw_az_g	Raw accelerometer XYZ in g
raw_gx_dps .. raw_gz_dps	Raw gyroscope XYZ in °/s
raw_roll_deg .. raw_yaw_deg	Raw euler angles in °
accel_lat_g .. accel_vert_g	Mount-corrected acceleration
gyro_x_dps .. gyro_z_dps	Mount-corrected gyro
pitch_deg, roll_deg, yaw_deg	Mount-corrected angles
mag_x, mag_y, mag_z	Magnetometer raw counts
temp_c	IMU die temperature

Legacy log warning

If the log was captured before timestamp prefixes were added to the logger, the parser will warn:

```
⚠  Legacy log detected (no per-frame timestamps).
   Upgrade to imu_logger2.2 for timestamped logs.
```

The data is still parsed correctly — you simply won't have per-frame timestamps.

---

12. Sensor Mount & Axis Correction

The WT901 is mounted **upside-down and flat** in the vehicle (the "-Z up" configuration — the sensor's Z axis points downward toward the ground). This is common when mounting in a tight enclosure. The sensor's firmware does not fully compensate for this, so axis corrections must be applied in software.

Physical orientation

```
Vehicle forward direction →

Top of enclosure (road-facing):  [normal bottom of PCB]
Bottom of enclosure (sky-facing): [normal top of PCB, connector side]

Sensor Z axis: points DOWN toward road (inverted from normal)
```

```
Sensor Y axis: longitudinal (forward/back), but INVERTED
Sensor X axis: lateral (left/right), OK
```

The raw roll reading in this mount is approximately -144° to -168°. This confirms the ~180° inversion.

Correction formulas

```
# Applied in witmotion_log_parser-2.py and imu_logger4_API.py

accel_lat   =  raw_ax           # lateral: no change
accel_lon   = -raw_ay           # longitudinal: inverted
accel_vert  = -raw_az           # vertical: inverted

gyro_x      =  raw_gx
gyro_y      = -raw_gy           # inverted with Y
gyro_z      =  raw_gz

pitch       = -(raw_roll + 180) # correct for 180° flip
roll        =  raw_pitch
yaw         =  raw_yaw          # heading unaffected by Z flip
```

Verification checklist

After installing in the car, verify corrections are correct by driving:

- **Hard braking:** `accel_lon` should go strongly positive (deceleration in forward direction).
- **Hard acceleration:** `accel_lon` should go negative.
- **Left turn:** `roll` should go positive (body rolls right = positive roll).
- **Flat road, straight:** `pitch` should be near 0°.
- **Uphill:** `pitch` should go positive.

If any of these are backwards, flip the sign on that axis correction.

---

13. Troubleshooting Log — Problems Solved

This section documents every significant issue encountered during development.

Problem: `witmotion-debug` not found when run via systemd

Symptom: Service starts, but immediately fails with `FileNotFoundError` or `command not found`.

Root cause: systemd runs with a minimal `PATH` that does not include the venv's `bin/` directory. The `witmotion-debug` binary lives in the venv at `/home/m1000/wt901/bin/witmotion-debug` and is not on the system PATH.

Fix: The `run_bootloader.sh` wrapper exports the venv bin directory to PATH:

```
export PATH="/home/m1000/wt901/bin:$PATH"
```

And the service file also sets it:

```
Environment="PATH=/home/m1000/wt901/bin:/usr/bin:/bin"
```

Both layers are needed for full coverage.

Problem: `RPi.GPIO` or `witmotion` not found despite being installed

Symptom: `ModuleNotFoundError` when running via systemd, but the script works fine when run manually.

Root cause: Running `python3 script.py` in the terminal uses the system Python or a different venv. The service must invoke the venv Python explicitly.

Fix: Always use the full absolute path to the venv Python in both the service file and any subprocess spawns:

```
PYTHON_BIN = "/home/m1000/wt901/bin/python"
```

Never use `python3` or `python` in systemd-launched scripts — they will resolve to the wrong interpreter.

Problem: Serial port silent — no data from IMU

Symptom: `witmotion-debug` runs but produces no output, or only `state: idle -> idle`.

Possible causes and fixes:

1. **Wrong port:** Try both `/dev/ttyAMA0` and `/dev/ttyS0`. Run `ls /dev/serial*` to see what's available.

2. **Serial console not disabled:** If the login shell is still attached to the UART, it will interfere. Re-run `raspi-config` and confirm the serial console is disabled.

3. **Bluetooth not disabled:** If `dtoverlay=disable-bt` is not in `config.txt`, the hardware UART is still assigned to Bluetooth and the GPIO pins are not exposed as a usable serial port.

4. **User not in `dialout` group:** Run `groups` to verify. Add with `sudo usermod -aG dialout m1000` then log out and back in.

5. **Wrong baud rate:** The WT901 defaults to 9600. Test with:

```
stty -F /dev/ttyAMA0 9600 raw
cat /dev/ttyAMA0 | xxd | head
```

You should see `0x55` bytes (the WT901 packet header).

Problem: Bricking the WT901 (device stops responding)

Symptom: The IMU powered on but produced no serial output at any baud rate.

Root cause: The WT901's baud rate was changed via a configuration command (either accidentally by a script or intentionally but then forgotten), and the host serial port was no longer configured to match.

Recovery procedure:

The WT901 can be recovered using the WitMotion PC software (Windows only) via USB-to-serial adapter, or by trying every possible baud rate in sequence:

```
for baud in 4800 9600 19200 38400 57600 115200 230400 460800; do
    echo "Trying $baud..."
    stty -F /dev/ttyAMA0 $baud raw
    timeout 2 cat /dev/ttyAMA0 | xxd | head -3
done
```

Once you find the baud rate that produces `0x55` bytes, you can send the command to reset it back to 9600 using the witmotion Python library:

```
from witmotion import IMU
imu = IMU(path='/dev/ttyAMA0', baudrate=<working_baud>)
imu.set_baudrate(9600)
```

Prevention: Do not change the WT901 baud rate unless you are certain you understand both the device-side and host-side configuration. If you do change it, update `SERIAL_PORT` and the `--baud` flag in the CMD list in `imu_logger2.1.py`.

Problem: `GPIO.cleanup()` errors on logger restart

Symptom: When the logger exits and the bootloader tries to re-initialize GPIO, warnings about channels already in use or cleanup failing.

Root cause: If the logger crashes (as opposed to exiting cleanly via the STOP button), GPIO may not be properly cleaned up.

Fix: The bootloader's `finally` block always calls `GPIO.cleanup()`. The re-initialization block after `logger_process.wait()` calls `GPIO.setmode(GPIO.BCM)` fresh. This pattern handles both clean exits and crashes.

Problem: Python venv breaks after OS upgrade or `apt upgrade`

Symptom: `ImportError` or `pip` errors after a system update.

Root cause: System Python version changed; the venv was built against the old version.

Fix: Recreate the venv from scratch:

```
cd /home/m1000/wt901
rm -rf bin lib include pyvenv.cfg
python3 -m venv .
./bin/pip install witmotion RPi.GPIO
```

Your scripts and data files are unaffected — only the venv directories are deleted.

Problem: `select()` read blocking the main loop

Symptom: Button presses are unresponsive during heavy data throughput.

Root cause: An earlier version of the logger used a blocking `readline()` call, which would hang if the subprocess paused or exited unexpectedly.

Fix: The current logger uses `select.select()` with a 0.1-second timeout:

```
ready, _, _ = select.select([process.stdout], [], [], 0.1)
if ready:
    line = process.stdout.readline()
else:
    continue
```

This guarantees the button check and LED blink run at least every 100ms.

Problem: Parsed CSV has no timestamps (all blank)

Symptom: The `timestamp_iso` and `time_hms` columns in the parsed CSV are empty.

Root cause: The log was captured with an older version of the logger that did not prepend `[HH:MM:SS.mmm]` timestamp prefixes to each line.

Fix: Upgrade to `imu_logger2.2+` which adds timestamps. Alternatively, accept that legacy logs will not have per-frame timestamps — the data values are still fully valid.

---

14. Calibration Procedure

The WT901 has onboard gyroscope and accelerometer calibration. This should be done when the sensor is first received, after any physical damage, or if angle drift is observed.

Using `imu_logger4_API.py` (interactive menu)

```
/home/m1000/wt901/bin/python imu_logger4_API.py
# Enter port: /dev/ttyS0 (or /dev/ttyAMA0)
# Enter baud: 9600
# Choose option 4: Calibrate accel/gyro
```

The script will count down and then issue the calibration command. Keep the sensor **completely still and level** for the full 5 seconds.

Manual calibration via witmotion library

```
from witmotion import IMU, CalibrationMode
imu = IMU(path='/dev/ttyAMA0', baudrate=9600)
imu.set_calibration_mode(CalibrationMode.AccelerometerGyroscopeCalibra
tion)
import time; time.sleep(5)
imu.set_calibration_mode(CalibrationMode.None_)
imu.close()
```

Notes on calibration

- Calibration results are stored in the WT901's non-volatile memory. You do not need to recalibrate after power cycling.
- The magnetometer requires a separate calibration (figure-8 rotation in 3D space). This is less critical for vehicle dynamics data but matters for heading/yaw accuracy.
- Calibrate before mounting the sensor in the car. Once mounted, level calibration is difficult.

---

15. Testing the System

Test 1: Manual run without systemd

Stop the service first, then run manually:

```
sudo systemctl stop wt901_bootloader.service
/home/m1000/wt901/bin/python /home/m1000/wt901/bootloadervw.py
```

BOOT_LED should turn on. ARMED_LED should blink. Press START — ARMED_LED should go solid briefly then off, and the logger should begin. Press STOP — logger should exit cleanly and the bootloader should return to blinking ARMED_LED.

Test 2: Check raw log output

```
tail -f /home/m1000/wt901/witmotion_raw.log
```

While logging, you should see a continuous stream of acceleration, gyroscope, angle, and magnetic messages.

Test 3: Run the parser

```
/home/m1000/wt901/bin/python /home/m1000/wt901/
witmotion_log_parser-2.py
```

The terminal will print a summary table. Verify the frame count, Hz rate (~10 Hz at default settings), and that the corrected angle values make sense for the sensor's current orientation.

Test 4: Service auto-start

```
sudo systemctl enable wt901_bootloader.service
sudo reboot
```

After reboot, BOOT_LED should turn on within ~15 seconds of power-on. If it does not, check the journal:

```
sudo journalctl -u wt901_bootloader.service -b
```

Test 5: Full drive test

Take the car out and log a short run with known maneuvers (hard stop, left turn, right turn, bump). Parse the log and verify that `accel_lon`, `roll`, and `pitch` all respond in the expected direction.

---

16. Future Improvements Roadmap

GPS Integration

Adding GPS provides ground truth position and velocity data, enabling speed-correlated analysis of g-forces, corner entry speed, etc.

Recommended hardware: Any UART-based GPS module such as the u-blox M8N, PA1010D, or similar. The Pi has a second UART available (`/dev/ttyAMA1`) after the primary is assigned to the WT901, or you can use a USB GPS dongle.

Integration approach:

```
GPS UART → /dev/ttyAMA1 (or USB) → gpsd daemon → Python gpsd client
```

Install `gpsd`:

```
sudo apt install gpsd gpsd-clients python3-gps
```

In the logger, poll `gpsd` in a separate thread and merge GPS coordinates into the CSV at each IMU frame. Key fields to capture: latitude, longitude, altitude, speed (m/s), GPS timestamp (for time synchronization).

Synchronization note: IMU timestamps come from `datetime.now()` on the Pi. GPS provides UTC time. If the Pi has no internet (typical in a car), the system clock drifts. A GPS-locked time sync via `gpsd` + `chrony` will keep timestamps accurate even without WiFi.

Live Telemetry Radio (LoRa or Cellular)

For real-time data transmission to a pit laptop, two approaches are practical at the SAE Baja scale:

Option A: LoRa radio (recommended for off-road use)

LoRa (Long Range) radio modules operate at 915 MHz in the US and achieve 1–10 km range even through terrain and buildings — ideal for Baja courses.

- Hardware: RFM95W modules, or purpose-built LoRa HATs (e.g., the WaveShare SX1268 HAT).
- Library: `lora-from-scratch` or the `pyLoRa` / `sx126x` Python libraries.
- Protocol: Send compressed JSON or binary-packed structs of the last 5 frames every 200ms. Do not send every frame — the LoRa data rate is low (~5 kbps at maximum range).

Option B: Cellular (LTE)

If your race area has cell coverage, a USB LTE dongle (e.g., Huawei E3372, or a Sixfab 4G HAT) provides higher bandwidth for near-real-time streaming.

- Stream to a cloud endpoint (simple Flask or FastAPI server).
- The Pi sends POSTed JSON payloads every 500ms.

General telemetry architecture:

```
imu_logger (thread) → telemetry_queue → radio_sender_thread → LoRa/LTE
```

Keep the telemetry sending in a separate daemon thread with a bounded queue. If the radio is busy or out of range, frames are dropped from the queue rather than backing up and crashing the logger.

Web Dashboard / Visualization

A live dashboard visible to pit crew on a laptop or tablet.

Approach: Run a lightweight web server on the Pi itself. The Pi serves a page over WiFi (either connected to a hotspot or running its own access point with `hostapd`).

Recommended stack:

- Backend: Python `FastAPI` or `Flask` with a WebSocket endpoint.

- Frontend: Plain HTML + JavaScript using Chart.js or similar for live graphs.
- The logger pushes new frames to a shared in-memory buffer; the WebSocket handler streams them to connected clients.

Minimal example architecture:

```
# In the logger, after writing each CSV row:
telemetry_buffer.append({
    "t": ts_ms,
    "ax": c_accel[0], "ay": c_accel[1], "az": c_accel[2],
    "roll": c_angle[1], "pitch": c_angle[0], "yaw": c_angle[2],
})

# In a separate FastAPI app (run in another thread or process):
@app.websocket("/ws")
async def websocket_endpoint(ws):
    while True:
        if telemetry_buffer:
            await ws.send_json(telemetry_buffer[-1])
        await asyncio.sleep(0.1)
```

Pi as WiFi access point: Useful when the car is out of WiFi range. Install `hostapd` and `dnsmasq`, configure the Pi to create a network called (e.g.) `BAJA-TELEMETRY`. Pit crew connect directly and browse to `http://192.168.4.1/`.

Additional Sensor Channels

With the serial and GPIO infrastructure in place, additional sensors are straightforward to add:

- **Suspension potentiometers:** Analog via ADS1115 (I2C ADC), 4 channels for 4-corner suspension travel.
- **Wheel speed sensors (hall effect):** GPIO interrupt-based pulse counting. RPM from pulse timing, speed from wheel circumference.
- **Engine RPM:** Tap into ignition signal, same approach as wheel speed via GPIO interrupt.
- **CAN bus (future):** If the vehicle gains a CAN-capable ECU, the Pi can read it via MCP2515 SPI CAN controller.

Post-Processing Improvements

- **Automatic session splitting:** The parser currently handles multi-session logs. Auto-split on `SESSION_START` markers for per-run CSV files.
- **G-force event detection:** Flag frames where `accel_lon` or `roll` exceeds a threshold — automatically highlight hard corners, jumps, and impacts.

- **Lap timing:** If a GPS waypoint is set for the start/finish, detect lap crossings and split the CSV into per-lap segments.

---

17. Quick Reference Cheat Sheet

Starting and stopping the service

```
sudo systemctl start wt901_bootloader.service
sudo systemctl stop wt901_bootloader.service
sudo systemctl restart wt901_bootloader.service
sudo systemctl status wt901_bootloader.service
sudo journalctl -u wt901_bootloader.service -f
```

Manual run (debug mode)

```
sudo systemctl stop wt901_bootloader.service
/home/m1000/wt901/bin/python /home/m1000/wt901/bootloadervw.py
```

Test IMU serial directly

```
/home/m1000/wt901/bin/witmotion-debug --path /dev/ttyAMA0 --verbose
```

Parse a log file

```
/home/m1000/wt901/bin/python /home/m1000/wt901/
witmotion_log_parser-2.py
# or with explicit paths:
/home/m1000/wt901/bin/python /home/m1000/wt901/
witmotion_log_parser-2.py witmotion_raw.log run1_parsed.csv
```

Reinstall venv from scratch

```
cd /home/m1000/wt901
rm -rf bin lib include pyvenv.cfg
python3 -m venv .
./bin/pip install witmotion RPi.GPIO
```

Check serial ports

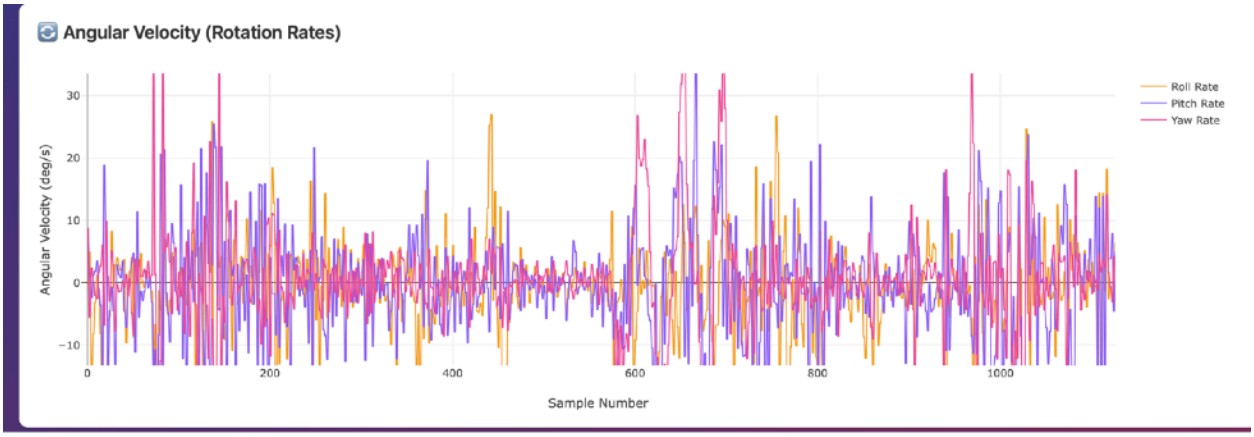
```
ls /dev/ttyAMA* /dev/ttyS* /dev/serial*
# Test data at 9600 baud:
stty -F /dev/ttyAMA0 9600 raw && timeout 3 cat /dev/ttyAMA0 | xxd |
head -10
```

GPIO pin reference (quick)

```
GPIO 13  → START button (other leg → GND)
GPIO 17  → STOP button  (other leg → GND)
GPIO 16  → SHUTDOWN button (other leg → GND)

GPIO 19  → ARMED LED    — 1 Hz blink = waiting for start
GPIO 21  → LOG LED      — 4 Hz blink = actively logging
GPIO 26  → BOOT LED     — solid = bootloader alive
GPIO 18  → SHUTDOWN LED — 10 Hz blink = shutdown in progress
```

```
GPIO 14 (TXD) → Yellow wire → WT901 RX
GPIO 15 (RXD) → Green wire  ← WT901 TX
GND terminal  → Purple wire → WT901 GND
5V terminal   → Red wire    → WT901 VCC
```

Document prepared March 2026. System built and validated on Raspberry Pi 4B (8 GB), Raspberry Pi OS Lite 64-bit, WitMotion WT901 firmware version 18159.

SAMPLE OUTPUTS: AFTER OFFLOAD -> ANALYSIS: